

15 SYSTEMS ENGINEERING MINI-PROJECTS CATALOG

Targeted, Weekly-Scale Technical Exercises with MNC Jira
Checklists

Scope: Low-level C, register-level FreeRTOS & Python HIL test scripts

Author: Senior Systems Architect & Coach

Workspace Root: /home/siva/sivaramireddy/Project1/

Minux Catalog: 15 High-Impact Systems Engineering Mini-Projects

This catalog presents **15 professional-grade, targeted systems mini-projects** (5 per track) designed to develop first-principles engineering proficiency. Each mini-project is structured exactly like standard MNC Jira task blocks, providing concise goals, technical stacks, and granular checklists.



Track 1: C Systems Programming (Mini-Projects 1–5)

1. Minux-Canary: Stack Frame Buffer Guard

- **Concept:** Implement a software-based stack buffer corruption detector that inserts compile-time custom patterns ("Canaries") before variables and asserts their integrity before returns.
- **Tech Stack:** C11, Assembly pointer auditing, GCC `-O0` debug context.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-C101]** Declare memory struct wrapping buffer space and trailing unsigned integer canary keys.
 - [] **[JIRA-MP-C102]** Write a macro `INIT_GUARD(obj)` that initializes random pattern keys inside structures.
 - [] **[JIRA-MP-C103]** Implement `int check_integrity(obj)` that compares canary memory offsets with initial patterns.
 - [] **[JIRA-MP-C104]** Test via intentional buffer overwrite strings to assert that corruption alerts trigger.

2. MemAlign-Light: Safe Block Memory Manager

- **Concept:** Build a fast memory allocator that pre-allocates an array of bytes and serves memory blocks aligned exactly to CPU word boundaries (4 or 8 bytes) to avoid unaligned access penalties.
- **Tech Stack:** C11, static assertions (`_Static_assert`), offset bitmasks math.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-C201]** Write alignment calculation macro: `#define ALIGN(sz) (((sz) + 7) & ~7)` (8-byte boundary).
 - [] **[JIRA-MP-C202]** Implement `void *mem_alloc(size_t size)` that shifts the offset pointer aligned to boundaries.
 - [] **[JIRA-MP-C203]** Add compile-time safety check ensuring total buffer capacity is a multiple of word sizes.

- [] [JIRA-MP-C204] Write test loops asserting that all returned pointers are multiples of 8.

3. BitRouter: Branch-Free Command Router

- **Concept:** Implement an ultra-fast event command router that processes packed 16-bit registers configurations and routes instructions to jump-tables without using any branch `if` or `switch` selectors.
- **Tech Stack:** Bitwise operations, function pointer arrays, address offsets mapping.
- **MNC Jira Tasks Checklist:**
- [] [JIRA-MP-C301] Define command structures mapping 4-bit operational codes and 12-bit payload values.
- [] [JIRA-MP-C302] Construct an array of 16 function pointers matching all possible 4-bit codes.
- [] [JIRA-MP-C303] Implement a branch-free execution loop: `handlers[opcode](payload)` executing instantly.
- [] [JIRA-MP-C304] Test and verify that passing invalid indexes returns safe, default "noop" handlers.

4. RawHex: Unbuffered Hexadecimal File Dump

- **Concept:** Write a low-level command-line hex dump tool that bypasses standard library buffers, reading binary files directly using unbuffered systems descriptors, and printing clean formatting strings.
- **Tech Stack:** Unbuffered system I/O (`open`, `read`), raw stdout descriptors.
- **MNC Jira Tasks Checklist:**
- [] [JIRA-MP-C401] Open binary targets using unbuffered `open(path, O_RDONLY)`.
- [] [JIRA-MP-C402] Read data into a fixed-size 16-byte buffer in a loop, handling partial block read returns cleanly.
- [] [JIRA-MP-C403] Write format formatting routines transforming bytes to hex chars (`0-F`) using bit-masks.
- [] [JIRA-MP-C404] Direct unbuffered write output strings to file descriptor `1` (`STDOUT_FILENO`).

5. SafeStr: Hardened MISRA-Compliant String Library

- **Concept:** Build a safe string library that replaces risky `strcpy` / `strcat` calls. The library must strictly audit buffer boundaries, allocate zero transient heap variables, and maintain 100% MISRA C compliance.
- **Tech Stack:** C11, static checkers (`cppcheck`), boundary assertions.
- **MNC Jira Tasks Checklist:**
- [] [JIRA-MP-C501] Implement `int safestr_copy(char *dest, size_t dest_sz, const char *src)` with bounds checking.

- [] **[JIRA-MP-C502]** Implement safe concatenation `safestr_concat()` tracking remaining buffer space.
- [] **[JIRA-MP-C503]** Add explicit null-termination guarantees on all exit paths, even under truncation.
- [] **[JIRA-MP-C504]** Verify clean cppcheck audits with zero style or safety standard alerts.



Track 2: Embedded Systems & Linux (Mini-Projects 6–10)

6. DMA-RingBuffer: UART Telemetry Stream

- **Concept:** Configure a UART transmitter on an STM32 MCU to continuously stream circular ring buffer telemetry using Direct Memory Access (DMA) channels, avoiding CPU processing interrupts.
- **Tech Stack:** Bare-metal MCU registers (USART, RCC, DMA), interrupt routines.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-E101]** Map a circular queue ring buffer struct inside microcontroller SRAM.
 - [] **[JIRA-MP-E102]** Configure DMA Stream registers mapping source to USART data register and enabling circular transfers.
 - [] **[JIRA-MP-E103]** Implement a thread-safe `enqueue` routine that feeds telemetry variables to the ring buffer.
 - [] **[JIRA-MP-E104]** Trace output bytes over a logic analyzer, validating that DMA handles stream transfers with zero CPU loads.

7. ADC-Alarm: Interrupt-Driven Voltage Guard

- **Concept:** Program an STM32 microcontroller to continuously read analog voltages from a potentiometer. If the value crosses an over-voltage limit, immediately trigger a hardware NVIC interrupt that de-energizes load pins.
- **Tech Stack:** STM32 register RCC/GPIO/ADC, NVIC interrupt vector setups.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-E201]** Initialize ADC registers configuring continuous scan mode and select channels.
 - [] **[JIRA-MP-E202]** Map ADC interrupt vector routines inside the custom startup vector table.
 - [] **[JIRA-MP-E203]** Implement the ADC ISR that checks voltages and pulls digital output pins immediately Low (0V).
 - [] **[JIRA-MP-E204]** Verify that the de-energize trigger fires in sub-microsecond intervals.

8. Watchdog-Sync: FreeRTOS Task Supervisor

- **Concept:** Build a dedicated high-priority watchdog supervisor task in FreeRTOS that pings lower-priority tasks and triggers a hardware Watchdog Reset if any task becomes unresponsive.

- **Tech Stack:** FreeRTOS Task API, Event Groups, Hardware Independent Watchdog (IWDG).

- **MNC Jira Tasks Checklist:**

- [] **[JIRA-MP-E301]** Initialize an STM32 Independent Watchdog (IWDG) register with a 1-second timeout.
- [] **[JIRA-MP-E302]** Create a FreeRTOS Event Group where each active task must set a unique bit periodically.
- [] **[JIRA-MP-E303]** Build the `Watchdog_Task` that blocks on event bits checks. If all bits are set, it kicks the watchdog.
- [] **[JIRA-MP-E304]** Test by forcing a task into an infinite sleep loop to verify that the hardware reboots successfully.

9. SysfsGPIO: Linux Device sysfs Driver

- **Concept:** Write an out-of-tree loadable Linux kernel module character driver that creates user-accessible control files in the `/sys/class/` directory, allowing users to toggle GPIOs via shell echoes.
- **Tech Stack:** Linux LKM, Sysfs nodes (`kobject`), GPIO Kernel API (`gpiod`).
- **MNC Jira Tasks Checklist:**
- [] **[JIRA-MP-E401]** Create a loadable kernel module register structure with class creation handlers.
- [] **[JIRA-MP-E402]** Create a sysfs attribute node named `gpio_state` mapping read/write callback functions.
- [] **[JIRA-MP-E403]** Implement the write callback to parse user string input (`"1"` or `"0"`) and drive GPIO registers.
- [] **[JIRA-MP-E404]** Verify that writing values (`echo 1 > /sys/class/minux/gpio_state`) toggles physical pins cleanly.

10. DTSTOverlay: Dynamic Device Tree Driver

- **Concept:** Compile and dynamically load a Device Tree Overlay (`.dtbo`) that adds a virtual telemetry peripheral node to your Embedded Linux board, and map a character driver to bind to it via compatible strings.
- **Tech Stack:** Device Trees (`DTS`), Device Tree Compiler (`DTC`), Linux platform driver interfaces.
- **MNC Jira Tasks Checklist:**
- [] **[JIRA-MP-E501]** Write a Device Tree Source file (`overlay.dts`) defining a virtual node with custom compat properties.
- [] **[JIRA-MP-E502]** Compile to overlay binary: `dtc -@ -I dts -O dtb -o overlay.dtbo overlay.dts`.
- [] **[JIRA-MP-E503]** Write a kernel platform driver that registers and binds to the device compatible string.

- [] [JIRA-MP-E504] Load overlay dynamically and verify that the kernel platform driver binds and probes successfully.
-

Track 3: Python Automation & HIL Frameworks (Mini-Projects 11–15)

11. AutoCal: SCPI VISA DMM Calibrator

- **Concept:** Design a Python script using PyVISA that connects to a digital multimeter (DMM) via SCPI commands, sweeps reference test voltages, calculates calibration offset coefficients, and writes them to the DMM.
- **Tech Stack:** Python 3, PyVISA, SCPI instrumentation mapping.
- **MNC Jira Tasks Checklist:**
 - [] [JIRA-MP-A101] Establish serial or ethernet VISA connection with a simulated DMM device.
 - [] [JIRA-MP-A102] Send SCPI query strings `:MEASURE:VOLTAGE:DC?` in a calibration loop.
 - [] [JIRA-MP-A103] Calculate drift variations and formulate linear offset equations.
 - [] [JIRA-MP-A104] Write calibration coefficients back to the DMM using safe write parameters.

12. stateless-HIL: Pytest Power Sweep

- **Concept:** Build a stateless test script in pytest using conftest fixtures that connects to a DC power supply, sweeps output voltage through parameterized values, and asserts output limits.
- **Tech Stack:** Python 3, pytest, parameterization decorators, instrument drivers.
- **MNC Jira Tasks Checklist:**
 - [] [JIRA-MP-A201] Write a conftest setup fixture that opens and resets a virtual DC power supply.
 - [] [JIRA-MP-A202] Create a parameterized sweep matrix: `@pytest.mark.parametrize("V", [1.0, 1.8, 3.3, 5.0])`.
 - [] [JIRA-MP-A203] Write the test case checking that measured values are within a 1% safety threshold.
 - [] [JIRA-MP-A204] Configure fixture teardown to de-energize output rails cleanly, even if the test assertions fail.

13. AsyncLogger: Thread-Safe Telemetry Log

- **Concept:** Construct a highly performant data-logger that spawns a dedicated consumer thread to pull telemetry packets off a synchronized queue and write them to a Parquet file, avoiding data loss.
- **Tech Stack:** Python 3, thread queues (`queue.Queue`), Pandas/PyArrow engine.
- **MNC Jira Tasks Checklist:**

- [] **[JIRA-MP-A301]** Build a consumer thread function loop that blocks on `queue.get()` with explicit exit checks.
- [] **[JIRA-MP-A302]** Implement a thread-safe telemetry producer pushing incoming mock values onto the queue.
- [] **[JIRA-MP-A303]** Inside the consumer, batch queue frames and write them portably into a Parquet dataset block.
- [] **[JIRA-MP-A304]** Verify 100% data preservation under high-frequency load stress tests.

14. ELFAudit: pre-Flash symbol checks

- **Concept:** Write a Python tool using `pyelftools` that parses a compiled ELF firmware file, searches for the exact memory address of a global telemetry buffer variable, and confirms it aligns to boundaries.
- **Tech Stack:** Python 3, `pyelftools`, symbol mappings.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-A401]** Open and parse compiled target ELF binary files in binary read mode.
 - [] **[JIRA-MP-A402]** Extract symbol tables (.symtab) and locate targeted variable keys.
 - [] **[JIRA-MP-A403]** Assert that the variable's memory offset address is divisible by 4 or 8 (proper alignment).
 - [] **[JIRA-MP-A404]** Integrate this check as a pre-flashing validation step, exiting with error code `-1` on alignment failures.

15. CI-Reporter: pytest JUnit notification

- **Concept:** Write a Python continuous integration script that executes a test suite, parses the resulting JUnit XML test report, formats a concise Markdown build status report, and posts it to a Slack webhook.
- **Tech Stack:** Python 3, pytest, JUnit XML ElementTree parsing, HTTP post requests.
- **MNC Jira Tasks Checklist:**
 - [] **[JIRA-MP-A501]** Execute pytest subprocess commands generating `report.xml` files.
 - [] **[JIRA-MP-A502]** Parse the generated XML extracting metrics: `tests`, `failures`, `skipped`, and execution time.
 - [] **[JIRA-MP-A503]** Formulate a clean Markdown dashboard string (e.g. Build Status: Passed. Tests: 12).
 - [] **[JIRA-MP-A504]** Dispatch this summary payload to a simulated Slack webhook URL using Python HTTP requests.